
The Cost of Senders for Coroutine I/O

Document Number: D4123R0
Date: 2026-03-17
Audience: LEWG, SG1
Reply-to: Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Concessions

3. Two Paths

3.1 Path A: Coroutine-Native Task

3.2 Path B: `std::execution::task<T, IoEnv>`

4. The Gap

5. The Gap Explained

5.1 Template Parameters

5.2 Sender Concept Instantiation

5.3 Task-to-Task `co_await` Cost

5.4 Completion Cost

5.5 The Combinator Gap

5.6 `io_env` Indirection

5.7 Stop Token Propagation

5.8 Implementation Complexity

6. The Schedule Risk

7. Conclusion

Acknowledgments

Abstract

Grant `std::execution::task` every proposed fix. Trace both paths. Show the gap.

This paper grants P3552R3^[1] every engineering fix that has been proposed or discussed, assumes they all ship, and traces two implementations of the same I/O coroutine side by side: one using a coroutine-native task with one template parameter, and one using `std::execution::task<T, IoEnv>` with two. The I/O hot path is identical. The gap is in the task-to-task machinery - the overhead that every `co_await` of a child task pays in the sender model but not in the coroutine-native model. The gap is small. It is not fatal. It is documented here with code from the `beman::task`^[2] reference implementation so the committee can evaluate it.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version. Supersedes D4051R0^[3].
-

1. Disclosure

The author developed and maintains Corosio^[4] and Capy^[5] and believes coroutine-native I/O is the correct foundation for networking in C++. The author provides information, asks nothing, and serves at the pleasure of the chair.

The author regards `std::execution` as an important contribution to C++ and supports its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper argues for removing, delaying, or diminishing `std::execution`.

2. The Concessions

This paper grants `std::execution::task` every engineering fix that has been proposed, discussed, or implied. The following are assumed to ship:

- I/O operations return awaitables, not senders. The template operation state problem (P4088R0^[6] Section 6.1) does not arise.
- Symmetric transfer works task-to-task. The stack overflow vulnerability (P3801R0^[7]) is resolved.
- Frame allocator timing is fixed. The rework in D3980R0^[8] ships. The allocator reaches `promise_type::operator new` before the frame is allocated.
- `AS-EXCEPT-PTR` does not convert routine `error_code` to `exception_ptr`. Routine I/O errors do not become exceptions.
- Compound results are handled inside the coroutine body via structured bindings.
`auto [ec, n] = co_await sock.read_some(buf)` works.
- `co_yield with_error` is unnecessary. Compound results stay in the coroutine body. The mechanism that P3801R0^[7] identified as blocking symmetric transfer is not needed.
- `IoEnv` is standardized as the networking environment, carrying a type-erased executor and a stop token.
- The promise delivers `IoEnv` to I/O awaitables through `await_transform`.
- Type-erased streams work under both models. Zero per-operation allocation.
- Separate compilation works under both models. I/O functions go in `.cpp` files.
- ABI stability works under both models. The vtable layout does not change.
- `when_all` and `when_any` provide structured concurrency under both models. Children complete before the parent resumes. Stop tokens propagate.
- Predicate-based combinators are achievable under both models. A custom `when_all` that inspects `set_value` arguments through a predicate can cancel siblings on I/O errors. Both models can build this.
- No performance gap on the I/O hot path. The awaitable, the syscall, the resumption, and the frame allocation are identical.
- Cross-domain bridges work. P4092R0^[9] and P4093R0^[10] demonstrate sender-to-coroutine and coroutine-to-sender interoperation.

Everything above is granted without reservation.

3. Two Paths

The same I/O operation. The same user code. Two task types.

3.1 Path A: Coroutine-Native Task

```
io::task<void> session(tcp_socket& sock)
{
    char buf[1024];
    auto [ec, n] = co_await sock.read_some(
        mutable_buffer(buf, sizeof buf));
    if (ec)
        co_return;
    co_await process(buf, n);
}
```

One template parameter. The promise carries `io_env` directly - a type-erased executor, a stop token, and an optional frame allocator. Constructed once at task start. Every I/O operation receives a pointer to it.

When `session` does `co_await process(buf, n)` - a child task - the awaiter stores the continuation handle and returns the child's handle. Symmetric transfer. Two pointer stores.

3.2 Path B: `std::execution::task<T, IoEnv>`

```
std::execution::task<void, IoEnv>
session(tcp_socket& sock)
{
    char buf[1024];
    auto [ec, n] = co_await sock.read_some(
        mutable_buffer(buf, sizeof buf));
    if (ec)
        co_return;
    co_await process(buf, n);
}
```

Two template parameters. The promise does not carry `io_env` directly. It holds a pointer to a `state_base` - an abstract base class with virtual functions - through which it reaches the scheduler, stop token, and environment.

When `session` does `co_await process(buf, n)`, the child task's `as_awaitable` returns an `awaiter<T, IoEnv, ParentPromise>`. The reference implementation^[2] does the following in `await_suspend`:

```

auto await_suspend(
    std::coroutine_handle<ParentPromise>
        parent) noexcept
{
    this->state_rep.emplace(
        env_receiver{&parent.promise()});
    this->scheduler.emplace(
        this->template
            from_env<scheduler_type>(
                get_env(parent.promise())));
    this->parent = std::move(parent);
    return this->handle.start(this);
}

```

Three operations before the child starts: construct a `state_rep` , extract the scheduler from the parent's environment, store the parent handle.

On completion, the `awaiter`^[2] checks whether the scheduler changed:

```

auto do_complete()
-> std::coroutine_handle<> override
{
    if constexpr (requires {
        *this->scheduler !=
            get_scheduler(get_env(
                this->parent.promise()));
    })
    {
        if (*this->scheduler !=
            get_scheduler(get_env(
                this->parent.promise())))
        {
            this->reschedule.emplace(
                this->parent.promise(),
                this);
            this->reschedule->start();
            return std::noop_coroutine();
        }
    }
    return this->actual_complete();
}

```

A scheduler comparison on every task completion. If the scheduler changed, a reschedule operation is constructed and started - a full sender `connect` / `start` cycle.

The promise reaches the scheduler through virtual dispatch. The `state_base`^[2] defines five virtual functions:

```
virtual auto do_complete()
    -> std::coroutine_handle<> = 0;
virtual auto do_get_stop_token()
    -> stop_token_type = 0;
virtual auto do_get_environment()
    -> Environment& = 0;
virtual auto do_get_scheduler()
    -> scheduler_type = 0;
virtual auto do_set_scheduler(
    scheduler_type other)
    -> scheduler_type = 0;
```

Every call to `get_scheduler()`, `get_stop_token()`, or `get_environment()` from the promise is an indirect call through a vtable.

4. The Gap

Property	Coroutine-native <code>task<T></code>	<code>task<T, IoEnv></code>
Type-erased streams	Identical	Identical
Separate compilation	Identical	Identical
ABI stability	Identical	Identical
Compound results in body	Identical	Identical
Symmetric transfer task-to-task	Identical	Identical
I/O hot path performance	Identical	Identical
Predicate-based combinators	Identical	Identical
Template parameters	1	2
Sender concept instantiation per task	0	1 per task type in chain

Property	Coroutine-native <code>task<T></code>	<code>task<T, IoEnv></code>
Task-to-task <code>co_await</code> cost	2 pointer stores	<code>state_rep</code> + scheduler extraction + handle
Task-to-task completion cost	Return parent handle	Scheduler comparison + potential reschedule
<code>io_env</code> access per I/O operation	Direct pointer	Virtual dispatch through <code>state_base</code>
Standard sender algorithms for I/O errors	Not applicable	Blind (custom variants possible)
<code>state_base</code> virtual functions in awaiter	0	5
Stop token propagation steps	5	17
Stop source bridges per <code>when_all</code>	0	2
Combinator algorithms needed for I/O	1	2 (standard + custom)
Implementation complexity (code lines)	210	1,271
Implementation complexity (types)	6	61
Implementation complexity (files)	1	27

The first seven rows are identical. The remaining rows are the gap.

5. The Gap Explained

5.1 Template Parameters

`task<T>` vs `task<T, IoEnv>`. Two spellings. A function returning one cannot be assigned to a variable of the other. Users must know which to use. Library interfaces must choose. Asio has lived with this for `awaitable<T, Executor>` for years. It is friction, not a structural problem.

5.2 Sender Concept Instantiation

Every `task<T, IoEnv>` satisfies the `sender` concept. The [reference implementation](#)^[2] declares:

```

using sender_concept =
    ::beman::execution::sender_t;

using xcompletion_signatures =
    ::beman::execution::detail::meta::combine<
        ::beman::execution::
            completion_signatures</* ... */>,
        ::beman::execution::
            completion_signatures<
                ::beman::execution::
                    set_stopped_t()>,
        error_types_of_t<Environment>>;

template<class Receiver>
auto connect(Receiver receiver) &&
    -> state<Receiver>;

```

Completion signatures, `connect`, and `state<Receiver>` are present in every task type. In a five-coroutine chain, five task types are instantiated, each carrying this machinery. None of the intermediate coroutines use it - they pass results via `co_await`, not via `connect` / `start`.

In the coroutine-native model, zero sender instantiations occur inside the chain. One bridge at the edge (P4093R0^[10]) connects the chain to the sender world.

5.3 Task-to-Task `co_await` Cost

In the coroutine-native model, `co_await child_task` stores the continuation handle and returns the child's handle. Two pointer stores. Symmetric transfer.

In the sender model, `co_await child_task` goes through `as_awaitable`, which returns an `awaiter<T, IoEnv, ParentPromise>`. The awriter's `await_suspend (source)`^[2]:

1. Constructs a `state_rep` (wraps an `env_receiver` pointing to the parent promise)
2. Extracts the scheduler from the parent's environment via `get_env(parent.promise())`
3. Stores the parent handle
4. Starts the child by passing itself as the `state_base*`

Per `co_await` of a child task. Multiplied by N in a chain of N coroutines.

5.4 Completion Cost

In the coroutine-native model, when a child task completes, the awaiter returns the parent's coroutine handle. Symmetric transfer. One pointer.

In the sender model, the awaiter's `do_complete (source)`^[2] compares the current scheduler against the parent's scheduler. If they differ, it constructs a reschedule operation (`awaiter_op_t`) that calls `execution::connect` and `execution::start` to transfer execution to the correct scheduler. This is a full sender `connect / start` cycle on the completion path.

When the scheduler has not changed - the common case for I/O - the comparison is a pointer comparison and the reschedule does not fire. The cost is the comparison itself, per task completion.

5.5 The Combinator Gap

A purported benefit of the unified sender model is that combinator algorithms like `when_all` need only be written once. One `when_all` serves all domains - GPU tasks, I/O tasks, timers. The coroutine-native model needs a second implementation. This is a real argument. This section examines whether it holds for I/O.

5.5.1 The `when_all` Completion Handler

P2300R10^[14] specifies `when_all`'s completion logic in `impls-for<when_all_t>::complete`. The handler dispatches on the completion channel tag:

```

[]<class Index, class State, class Rcvr,
    class Set, class... Args>(
    this auto& complete,
    Index, State& state, Rcvr& rcvr,
    Set, Args&&... args) noexcept
-> void
{
    if constexpr (
        same_as<Set, set_error_t>)
    {
        if (disposition::error !=
            state.disp.exchange(
                disposition::error))
        {
            state.stop_src.request_stop();
            TRY-EMPLACE-ERROR(state.errors,
                forward<Args>(args)...);
        }
    }
    else if constexpr (
        same_as<Set, set_stopped_t>)
    {
        auto expected =
            disposition::started;
        if (state.disp
            .compare_exchange_strong(
                expected,
                disposition::stopped))
        {
            state.stop_src.request_stop();
        }
    }
    else if constexpr (
        !same_as<decltype(State::values),
            tuple<>>)
    {
        if (state.disp ==
            disposition::started)
        {
            auto& opt =
                get<Index::value>(
                    state.values);
            TRY-EMPLACE-VALUE(complete,
                opt,
                forward<Args>(args)...);
        }
    }
}

```

```

    }

    state.arrive(rcvr);
}

```

Three branches. Three behaviors:

- `set_error` : calls `stop_src.request_stop()` . Siblings are cancelled. The error is stored.
- `set_stopped` : calls `stop_src.request_stop()` . Siblings are cancelled.
- `set_value` (the `else` branch): stores the values. No inspection. No stop request. Siblings keep running.

The `set_value` branch unconditionally stores values and calls `state.arrive(rcvr)` . There is no mechanism in the specification to inspect value-channel arguments, apply a predicate, or decide to cancel based on the values received.

5.5.2 Three Routing Options

An I/O read returns `(error_code, size_t)` . Three ways to route it through the sender channels:

Routing	Preserves	Breaks
<code>set_value(ec, n)</code> - both values on the value channel	Both values	<code>when_all</code> does not cancel siblings, <code>upon_error</code> unreachable, <code>retry</code> does not fire
<code>set_error(ec)</code> on failure, <code>set_value(n)</code> on success	Composition algebra	Byte count destroyed on error - partial reads lost
<code>set_value(pair{ec, n})</code> - pair on the value channel	Both values	Same as row 1 - composition blind to errors

Kohlhoff identified this in 2021:

“Due to the limitations of the `set_error` channel (which has a single ‘error’ argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel.” (P2430R0^[15])

5.5.3 The Trade-Off Table

The sender community must choose which set of costs to accept:

Option	Sibling cancellation	Byte count preserved	Standard <code>when_all</code> works	Custom algorithm needed
Route through <code>set_error</code>	Yes	No	Yes	No
Route through <code>set_value</code>	No	Yes	Yes	Yes (for cancellation)
Write custom <code>when_all</code>	Yes	Yes	No	Yes

Every sender option either loses data (the byte count) or requires a custom algorithm. If the byte count matters - and in networking it does, because partial reads are routine - the standard `when_all` is insufficient. A custom `when_all` whose internal receiver inspects `set_value` arguments is needed. Both models can build this custom algorithm.

The “write it once” argument does not survive contact with compound I/O results. The sender model needs two `when_all` implementations for I/O: the standard one for channel-dispatched cancellation and a custom one for value-inspecting cancellation.

5.5.4 The Coroutine-Native Path

In the coroutine-native model, the runner coroutine inside `when_all` has the full result after `co_await` :

```

auto result =
    co_await std::move(inner);
// result is io_result{ec, n}
// Both values are here.

// Store first. Always preserve.
std::get<Index>(
    state->results_).set(result);

// Then decide whether to cancel.
if (result.ec)
    state->core_
        .stop_source_.request_stop();

```

No channel routing. No data loss. The runner inspects the result value directly. The child coroutine does not know about the cancellation policy - the combinator applies it after the child returns.

The coroutine-native model does not have a parallel set of standard channel-dispatched algorithms that remain blind. There is one `when_all`. It can inspect the result because the runner coroutine has the value, not just the channel tag.

5.6 `io_env` Indirection

In the coroutine-native model, the promise carries `io_env` directly. The type-erased executor is constructed once at task start. Every I/O operation receives a pointer to it. No indirection.

In the sender model, the promise holds a `state_base*` pointer. Reaching the scheduler requires a virtual call: `get_state()->do_get_scheduler()`. The `state_base`^[2] has five virtual functions. Every query from the promise - scheduler, stop token, environment - is an indirect call.

The cost per call is small - a pointer dereference and an indirect function call. The cost is paid on every I/O operation that needs the executor (to submit to the reactor) and on every task-to-task transition (to extract the scheduler for the child).

5.7 Stop Token Propagation

Cancellation in `when_all` requires propagating a stop signal from the shared stop source to each child's I/O operations. The two models achieve this through different paths.

Coroutine-native model (5 steps):

1. `when_all_core` creates a `stop_source_`.
2. `launch_one` passes `stop_source_.get_token()` in `io_env` to each child.
3. The child's I/O awaitable reads `env->stop_token` directly.
4. On error, the runner calls `stop_source_.request_stop()`.
5. Siblings see `stop_requested()` in their `io_env.stop_token`.

One hop from stop source to child. No bridging. No virtual calls. The `io_env` carries the token directly.

Sender model (`task<T, IoEnv>` , all fixes granted, 17 steps):

1. `when_all` creates a stop source.
2. `when_all` connects each child task to a receiver carrying the stop token in its environment.
3. `task::connect` creates `state<Receiver>` storing the receiver.
4. `state` bridges the receiver's stop token to its own stop source via a `stop_callback`.
5. `state` implements `do_get_stop_token()` as a virtual function on `state_base`.
6. The child promise calls `get_stop_token()`, which dispatches through the virtual `do_get_stop_token()`.
7. `await_transform` constructs `io_env` from the `get_stop_token()` return value.

8. `await_transform` wraps the I/O awaitable in `io_wrapper` , passing the `io_env` .
9. The I/O awaitable reads the stop token from `io_env` .
10. On the task-to-task `co_await` path: the `awaiter` must also bridge the stop token.
11. The `awaiter` implements its own `do_get_stop_token()` override.
12. The `awaiter` creates its own `stop_callback` bridging the parent's token.
13. On error: the custom receiver in `when_all` inspects `set_value(ec, n)` .
14. The receiver calls `stop_source_.request_stop()` .
15. The stop request propagates through step 4's bridge to each child's stop source.
16. Each child's `do_get_stop_token()` returns the bridged token.
17. The I/O awaitable sees `stop_requested()` .

Property	Coroutine-native	Sender (all fixes)
Steps to propagate stop	5	17
Virtual calls per propagation	0	2
Stop source bridges	0	2
Stop callbacks constructed	0	2

The current `beman::task`^[2] reference implementation's `awaiter::do_get_stop_token()`^[2] returns a default-constructed stop token - the bridge on the `co_await` path is not implemented. This paper grants the fix. The bug demonstrates the complexity cost: more machinery means more places to get it wrong.

5.8 Implementation Complexity

The `beman::task`^[2] reference implementation of `P3552R3`^[1] consists of 27 header files in `include/beman/task/detail/` . Lines below are code only - blank lines, comment-only lines, and documentation lines are excluded:

File	Code lines
<code>result_type.hpp</code>	146
<code>task_scheduler.hpp</code>	144
<code>promise_type.hpp</code>	103
<code>awaiter.hpp</code>	90

File	Code lines
<code>into_optional.hpp</code>	84
<code>task.hpp</code>	69
<code>poly.hpp</code>	61
<code>allocator_support.hpp</code>	56
<code>state.hpp</code>	54
<code>state_base.hpp</code>	42
<code>state_rep.hpp</code>	38
<code>inline_scheduler.hpp</code>	36
<code>handle.hpp</code>	36
<code>promise_base.hpp</code>	34
<code>logger.hpp</code>	30
<code>find_allocator.hpp</code>	29
<code>promise_env.hpp</code>	28
<code>change_coroutine_scheduler.hpp</code>	25
<code>allocator_of.hpp</code>	23
<code>single_thread_context.hpp</code>	22
<code>sub_visit.hpp</code>	20
<code>scheduler_of.hpp</code>	19
<code>error_types_of.hpp</code>	18
<code>stop_source.hpp</code>	17
<code>completion.hpp</code>	16
<code>final_awaiter.hpp</code>	15
<code>with_error.hpp</code>	14
Total	1,271

The implementation defines 61 distinct class, struct, and enum types across these files.

A coroutine-native task that provides the same I/O capabilities - symmetric transfer, compound results, stop token propagation, frame allocator support - is a single file with 210 lines of code and 6 types.

Metric	<code>std::execution::task</code>	Coroutine-native task
Header files	27	1
Code lines	1,271	210
Distinct types	61	6

The difference is not in what the task does for the user. Both provide the same I/O capabilities (Section 2). The difference is in the sender machinery that the coroutine-native task does not carry: `state_base` with five virtual functions, `state_rep`, `awaiter_op_t` for rescheduling, `completion` for channel routing, `into_optional` for sender result adaptation, `poly` for type erasure of the scheduler, `inline_scheduler`, `promise_env` for environment forwarding, and `error_types_of` for completion signature computation.

6. The Schedule Risk

The concessions in Section 2 assume six engineering fixes ship. [P3552R3](#)^[1] is the vehicle. The C++26 cycle is closing. The fixes are not in [P3552R3](#)^[1] today:

- Symmetric transfer: [P3801R0](#)^[7] identified the vulnerability. Trampolines are being explored ([P3796R1](#)^[11]). Not landed.
- Frame allocator timing: [D3980R0](#)^[8] is in progress. Not landed.
- `IoEnv` : does not exist in any proposal.
- Error delivery: `AS-EXCEPT-PTR` is still in the specification.

If `task` ships in C++26 without these fixes, the concessions in Section 2 are hypothetical. The gap in Section 4 would be larger - the first seven rows would no longer say “Identical.”

[P2762R0](#)^[12] mentioned `io_task` in one paragraph:

“It may be useful to have a coroutine task (`io_task`) injecting a scheduler into asynchronous networking operations used within a coroutine... The corresponding task class probably needs to be templated on the relevant scheduler type.”

P2762^[13] stopped at R2 (October 2023). No revision in over two years. No published paper defines what `IoEnv` looks like inside `std::execution::task`.

The implementation section of P3552R3^[1] acknowledges:

“This implementation hasn’t received much use, yet, as it is fairly new.”

7. Conclusion

The gap is small. It is not fatal. The I/O hot path is identical. The difference is in the task-to-task machinery: `state_rep` construction, virtual dispatch for scheduler access, scheduler comparison on completion, and five virtual functions in the awaiter’s `state_base`. In the coroutine-native model, the same path is two pointer stores and a symmetric transfer.

The committee has the information to evaluate whether the gap justifies a separate task type for I/O. This paper provides the evidence. The committee decides.

Acknowledgments

The author thanks Dietmar Kühl for P3552R3 and P3796R1 and for `beman::execution`; Jonathan Müller for P3801R0 and for documenting the symmetric transfer gap; Chris Kohlhoff for identifying the partial-success problem in P2430R0; Eric Niebler, Lewis Baker, and Kirk Shoop for `std::execution`; Steve Gerbino for co-developing the bridge implementations; Peter Dimov for the frame allocator propagation analysis; and Mungo Gill, Mohammad Nejati, and Michael Vandenberg for feedback.

References

1. P3552R3 - “Add a Coroutine Task Type” (Dietmar Kühl, Maikel Nadolski, 2025). <https://wg21.link/p3552r3>
2. `bemanproject/task` - P3552R3 reference implementation. <https://github.com/bemanproject/task>
3. D4051R0 - “Steelmanning P3552R3” (Vinnie Falco, 2026). <https://wg21.link/d4051r0>
4. `cppalliance/corosio` - Coroutine-native networking library. <https://github.com/cppalliance/corosio>

5. [cppalliance/capy](https://github.com/cppalliance/capy) - Coroutine I/O primitives library. <https://github.com/cppalliance/capy>
6. [P4088R0](https://isocpp.org/files/papers/P4088R0.pdf) - "The Case for Coroutines" (Vinnie Falco, 2026). <https://isocpp.org/files/papers/P4088R0.pdf>
7. [P3801R0](https://wg21.link/p3801r0) - "Concerns about the design of `std::execution::task`" (Jonathan Müller, 2025).
<https://wg21.link/p3801r0>
8. [D3980R0](https://isocpp.org/files/papers/D3980R0.html) - "Task's Allocator Use" (Dietmar Kühl, 2026). <https://isocpp.org/files/papers/D3980R0.html>
9. [P4092R0](https://isocpp.org/files/papers/P4092R0.pdf) - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026).
<https://isocpp.org/files/papers/P4092R0.pdf>
10. [P4093R0](https://isocpp.org/files/papers/P4093R0.pdf) - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026).
<https://isocpp.org/files/papers/P4093R0.pdf>
11. [P3796R1](https://wg21.link/p3796r1) - "Coroutine Task Issues" (Dietmar Kühl, 2025). <https://wg21.link/p3796r1>
12. [P2762R0](https://wg21.link/p2762r0) - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). <https://wg21.link/p2762r0>
13. [P2762R2](https://wg21.link/p2762r2) - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). <https://wg21.link/p2762r2>
14. [P2300R10](https://wg21.link/p2300r10) - "std::execution" (Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). <https://wg21.link/p2300r10>
15. [P2430R0](https://wg21.link/p2430r0) - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). <https://wg21.link/p2430r0>